

Manual de Instalação e Operação

Agentic Customer Intelligence

Gabriel Cravalho Domingos da Conceição

Maio de 2026

1 Objetivo

Este manual descreve os componentes necessários para instalar, configurar e executar localmente o ambiente do projeto Agentic Customer Intelligence. O projeto contém pipeline de clusterização, backend RAG de personas, persistência em banco de dados, interface web e integração com uma API local compatível com OpenAI.

2 Pré-requisitos

- Linux ou ambiente compatível com shell Bash.
- Python 3.11 ou superior.
- uv instalado.
- Docker e Docker Compose.
- Node.js 20 ou superior.
- npm.
- API local compatível com OpenAI para chat e embeddings.
- Opcional: Jupyter Notebook ou JupyterLab para executar o notebook de ML.

3 Estrutura Principal do Projeto

```
agentic-customer-intelligence/  
|-- data/
```

```
|-- ml-clustering/  
|-- agentic-personas/  
|   |-- backend/  
|   |-- knowledge_base/  
|   '-- src/  
|-- apps/web/  
|-- infra/  
|-- docs/  
|-- docker-compose.yml  
|-- pyproject.toml  
'-- uv.lock
```

4 Componentes do Ambiente

- **PostgreSQL + pgvector:** armazena embeddings e chunks da base de conhecimento.
- **MongoDB:** armazena conversas e histórico de mensagens.
- **Backend FastAPI 8010:** API principal das personas e do chat RAG.
- **Frontend React 5173:** interface web para chat coletivo e individual.
- **API agentic 8020:** API auxiliar do pacote `agentic-personas`.
- **LLM local 8000:** serviço compatível com OpenAI, usado para chat e embeddings.

5 Configuração Inicial

5.1 Instalar dependências Python

Na raiz do projeto:

```
uv sync
```

5.2 Configurar variáveis de ambiente

Copie os arquivos de exemplo:

```
cp .env.example .env  
cp agentic-personas/backend/.env.example agentic-personas/backend/.env  
cp apps/web/.env.example apps/web/.env
```

Principais variáveis:

```
OPENAI_BASE_URL=http://127.0.0.1:8000/v1
OPENAI_API_KEY=local
OPENAI_MODEL=qwen-linkedin
OPENAI_EMBEDDING_MODEL=embed-local
EMBEDDING_DIM=768
```

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=agentic_customer_intelligence
POSTGRES_USER=aci
POSTGRES_PASSWORD=aci
```

```
MONGO_HOST=localhost
MONGO_PORT=27017
MONGO_DB=agentic_customer_intelligence
MONGO_COLLECTION=conversations
```

```
VITE_API_BASE_URL=http://127.0.0.1:8010/api
```

Observação: `EMBEDDING_DIM` deve ser compatível com a dimensão retornada pelo serviço local de embeddings.

6 Subir Bancos de Dados

Na raiz do projeto:

```
docker-compose up -d mongo postgres
```

Verificação:

```
docker ps
```

Serviços esperados:

- aci-mongo em localhost:27017;
- aci-postgres em localhost:5432.

7 Subir a API Local de LLM

O backend espera um serviço compatível com OpenAI, expondo:

```
POST /v1/chat/completions
```

```
POST /v1/embeddings
```

Exemplo de verificação:

```
curl http://127.0.0.1:8000/v1/models \  
-H "Authorization: Bearer local"
```

Este projeto não fixa uma implementação única do LLM. Pode-se usar uma API local baseada em llama.cpp, vLLM, Ollama com proxy compatível, ou outro servidor que respeite o formato OpenAI.

8 Executar Pipeline de Machine Learning

Para regenerar perfis de cluster:

```
PYTHONPATH=ml-clustering/src \  
uv run python ml-clustering/run_pipeline.py
```

Para gerar resumo executivo da parte de ML:

```
PYTHONPATH=ml-clustering/src \  
uv run python ml-clustering/generate_part1_summary.py
```

Para abrir o notebook Jupyter:

```
uv run jupyter notebook \  
ml-clustering/notebooks/customer_segmentation_clustering.ipynb
```

Os principais outputs são escritos em:

```
ml-clustering/outputs/
```

9 Ingerir Base de Conhecimento RAG

Com PostgreSQL, pgvector e API de embeddings ativos:

```
PYTHONPATH=agentic-personas/backend \  
uv run python agentic-personas/backend/scripts/ingest_knowledge_base.py
```

Esse script recria a tabela vetorial:

```
persona_knowledge_chunks
```

10 Subir Backend Principal

Na raiz do projeto:

```
cd agentic-personas/backend
PYTHONPATH=/caminho/para/agentic-customer-intelligence/agentic-personas/backend \
uv run uvicorn app.main:app --host 0.0.0.0 --port 8010 --reload
```

Substitua `/caminho/para/...` pelo caminho local do projeto.

Verificação:

```
curl http://127.0.0.1:8010/api/health
```

Resposta esperada:

```
{"status":"ok","app_name":"Agentic Customer Intelligence API"}
```

11 Subir Frontend

Instale dependências na primeira execução:

```
cd apps/web
npm install
```

Execute o servidor Vite:

```
npm run dev -- --host 0.0.0.0 --port 5173 --strictPort
```

Acesse:

```
http://localhost:5173
```

O uso de `--strictPort` é recomendado para evitar que o Vite suba silenciosamente em outra porta caso a porta 5173 esteja ocupada.

12 Subir API Agentic Auxiliar

Na raiz do projeto:

```
PYTHONPATH=/caminho/para/agentic-customer-intelligence/agentic-personas/src \
uv run uvicorn agentic_personas.api:app --host 0.0.0.0 --port 8020 --reload
```

Esse componente é auxiliar e pode ser usado para exploração do pacote `agentic_personas`.

13 Comando Integrado de Desenvolvimento

Um fluxo típico de desenvolvimento envolve três terminais:

1. bancos e LLM local;
2. backend principal na porta 8010;
3. frontend na porta 5173.

Exemplo:

```
docker-compose up -d mongo postgres
```

```
cd agentic-personas/backend
PYTHONPATH=$(pwd) uv run uvicorn app.main:app \
  --host 0.0.0.0 --port 8010 --reload
```

```
cd ../../apps/web
npm run dev -- --host 0.0.0.0 --port 5173 --strictPort
```

14 Guia Prático de Uso da Ferramenta

Após subir os serviços, abra <http://localhost:5173>. O fluxo prático de uso é:

1. Criar uma conversa com o botão **Nova conversa**.
2. Escolher **Chat coletivo com as 3 personas** para comparação simultânea.
3. Ou clicar em **Conversar com esta persona** em um card para chat individual.
4. Digitar a pergunta no campo inferior e enviar no botão de envio.

Botões e ações principais na interface:

- **Nova conversa**: cria um novo contexto de chat.
- **Limpar tudo**: remove o histórico local de conversas da barra lateral.
- **Regenerate**: gera novamente a última resposta com o mesmo prompt.
- **Edit + Resend**: permite editar a pergunta e reenviar.
- **Limpar**: limpa mensagens da conversa atual.

- **Excluir:** remove a conversa atual.

Guia rápido para usar melhor:

- Perguntas de comparação: use chat coletivo.
- Perguntas táticas por segmento: use chat individual.
- Perguntas de preço e promoção: peça contraste explícito entre as três personas.
- Para sessões de demo: inicie por uma pergunta ampla e depois aprofunde por persona.

15 Menu Run and Debug (VS Code)

Se o workspace estiver com as configurações de execução carregadas, o menu *Run and Debug* expõe atalhos para subir e parar os serviços sem usar terminal manualmente.

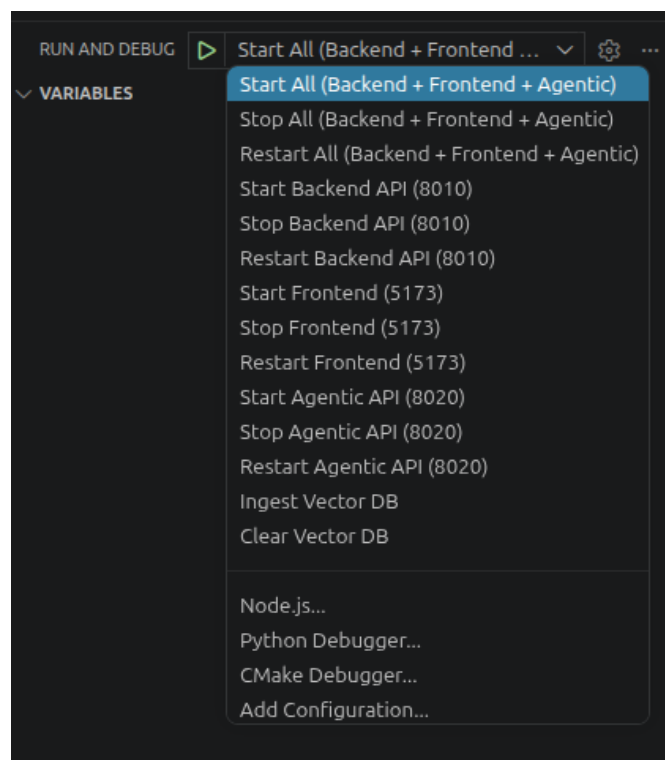


Figure 1: Menu Run and Debug com atalhos de orquestracao local

15.1 Atalhos combinados

- **Start All (Backend + Frontend + Agentic):** inicia backend 8010, frontend 5173 e API agentic 8020.

- Stop All (Backend + Frontend + Agentic): encerra os três serviços.
- Restart All (Backend + Frontend + Agentic): reinicia os três serviços.

15.2 Atalhos por serviço

- Start Backend API (8010) / Stop Backend API (8010) / Restart Backend API (8010).
- Start Frontend (5173) / Stop Frontend (5173) / Restart Frontend (5173).
- Start Agentic API (8020) / Stop Agentic API (8020) / Restart Agentic API (8020).

15.3 Ações de base vetorial

- Ingest Vector DB: executa ingestão dos arquivos da base de conhecimento no PostgreSQL/pgvector.
- Clear Vector DB: limpa os dados vetoriais previamente ingeridos.

15.4 Fluxo recomendado com os botões

1. Executar Start All (Backend + Frontend + Agentic).
2. Executar Ingest Vector DB ao atualizar conteúdos em `knowledge_base`.
3. Abrir `http://localhost:5173` e validar a tela inicial.
4. Ao finalizar, executar Stop All (Backend + Frontend + Agentic).

16 Endpoints Úteis

- GET `/api/health`: status da API.
- GET `/api/personas`: lista personas.
- GET `/api/personas/{cluster_id}`: detalhe de persona.
- POST `/api/chat`: chat não-streaming.
- POST `/api/chat/stream`: chat streaming.
- POST `/api/retrieval/test`: teste de recuperação RAG.

- POST /api/conversations: cria conversa.
- GET /api/conversations: lista conversas.
- PATCH /api/conversations/{id}: atualiza título.
- DELETE /api/conversations/{id}: exclui conversa.
- DELETE /api/conversations/{id}/messages: limpa mensagens.

17 Validação de Qualidade

Para executar a bateria de avaliação das respostas:

```
uv run python agentic-personas/backend/scripts/evaluate_persona_quality.py
```

Arquivos gerados:

```
agentic-personas/backend/outputs/persona_answer_battery.json
docs/persona_answer_battery_report.md
```

18 Build de Produção do Frontend

```
cd apps/web
npm run build
```

O resultado é gerado em:

```
apps/web/dist/
```

Esse diretório é artefato de build e não deve ser versionado.

19 Limpeza e Solução de Problemas

19.1 Porta do Vite ocupada

Listar processos:

```
lsof -iTCP:5173 -sTCP:LISTEN -n -P
```

Encerrar processo:

```
kill <PID>
```

19.2 Backend não responde

Verificar portas:

```
lsof -iTCP:8010 -sTCP:LISTEN -n -P
```

Verificar health:

```
curl http://127.0.0.1:8010/api/health
```

19.3 Erro de embeddings

Verifique:

- se a API local de LLM está ativa;
- se `OPENAI_BASE_URL` está correto;
- se `OPENAI_EMBEDDING_MODEL` existe no servidor local;
- se `EMBEDDING_DIM` corresponde ao vetor retornado.

19.4 Recriar bancos

Atenção: o comando abaixo remove volumes persistidos.

```
docker-compose down -v
```

```
docker-compose up -d mongo postgres
```

20 Ordem Recomendada para Demo

1. Subir MongoDB e PostgreSQL.
2. Subir API local de LLM e embeddings.
3. Rodar ingestão da base de conhecimento.
4. Subir backend principal em 8010.
5. Subir frontend em 5173.
6. Abrir `http://localhost:5173`.
7. Usar chat coletivo ou cards de personas individuais.